Capítulo 1

Introducción

1.1. Motivación

En las ciencias de la computación, las tecnologías avanzan rápidamente, y los sistemas antiguos suelen quedar obsoletos; sin embargo, ocasionalmente surge la necesidad de recuperar el uso de máquinas históricas, motivados por el interés histórico o educativo.

Durante la segunda mitad de los años 80 y la primera de los 90, la industria del entretenimiento experimentó un notable crecimiento debido a la proliferación de fabricantes de hardware a nivel mundial. Ordenadores domésticos como el ZX Spectrum o el Amstrad CPC llegaron a los hogares europeos, mientras que videoconsolas de 8 bits, basadas en procesadores como el Z80 o el 68000 y procedentes de Asia, permitieron que muchos usuarios tuvieran sus primeros contactos con software desarrollado, como la Nintendo Entertainment System o la portátil Game Boy.

El desarrollo de software para estas plataformas requiere conocimientos específicos de sus arquitecturas y limitaciones. En aquella época, los recursos eran reducidos y la mayoría del software se programaba en ensamblador directamente para el procesador objetivo, sin herramientas modernas de desarrollo.

En la actualidad, aunque existen compiladores para estas máquinas, su uso requiere gestionar herramientas dispersas y un conocimiento avanzado del entorno, lo que limita la accesibilidad y aumenta la complejidad del desarrollo.

El desarrollo enfocado a estas plataformas, además tiene ventajas formativas de cara a la preparación de futuros desarrolladores ya que centra el desarrollo de software en la optimización de recursos disponibles y la programación cercana al hardware.

Otro aspecto relevante es la preservación del patrimonio cultural digital. Aunque actualmente puede encontrarse información sobre sistemas históricos en internet, esta se halla dispersa y depende en gran medida de iniciativas individuales y voluntarias. La falta de centralización y de mecanismos formales de conservación implica el riesgo de que, a medio o largo plazo, parte del conocimiento técnico asociado a estas arquitecturas, así como el software desarrollado para ellas, pueda desaparecer. En este sentido, el presente proyecto contribuye a la preservación de esta cultura tecnológica mediante la promoción de la reutilización de estas plataformas, el desarrollo de nuevo software y la adaptación parcial de aplicaciones a entornos actuales.

1.2. Objetivos

El objetivo de este trabajo es crear un entorno de desarrollo integrado (IDE) en Java que permita el desarrollo de software en C para sistemas de 8 bits. Entre sus características principales se incluyen la compilación cruzada, un sistema de plugins para ampliar funcionalidades y la posibilidad de añadir nuevos compiladores.

Objetivos específicos:

- Implementar integración automática con compiladores cruzados.
- Diseñar una arquitectura modular basada en plugins.
- Permitir la incorporación de nuevas plataformas sin modificar el núcleo del sistema.
- Simplificar el flujo de trabajo del desarrollador de software clásico.

El alcance del proyecto se centra principalmente en las plataformas Game Boy y ZX Spectrum, dado que existen librerías ya desarrolladas (GBDK [**?**] y Z88DK [**?**]) y el software para estas máquinas es de pequeño tamaño, facilitando la realización de pruebas y la validación de la herramienta. Otra gran ventaja de usar estos sistemas es que utilizan arquitecturas basadas total o parcialmente en el procesador Z80 desarrollado por Zilog. Asimismo, se ha diseñado el sistema de plugins para permitir futuras ampliaciones del IDE y facilitar la inclusión de más sistemas desarrollables dentro de la herramienta.

Justificación técnica de la selección de plataformas Gameboy y ZX Spectrum:

- Ambas usan CPU basadas en Z80 (total o parcialmente).
- Comunidad activa.
- Librerías existentes.
- Tamaño reducido de ROM $\rightarrow$ validación rápida.

Los capítulos siguientes presentan la metodología utilizada, el desarrollo de la herramienta, las pruebas realizadas, los resultados obtenidos y las conclusiones derivadas de este proyecto.

1.3. Estado del arte

En este apartado se analizarán los ejemplos más representativos de herramientas para el desarrollo de aplicaciones de 8 bits. Se incluirán tanto IDEs como compiladores.

La Tabla 1.1 recoge un resumen comparativo de las principales características de los entornos de desarrollo más representativos que existen en la actualidad, que serán analizadas en detalle en las subsecciones posteriores. Se prestará especial atención a distinguir entre aquellas funcionalidades integradas de forma nativa y aquellas que dependen de complementos externos o configuraciones adicionales.

Característica	**GBDK**	**Z88DK**	**ZX Basic Studio**	**VS Code + plugins**	**Eclipse CDT + toolchains**
IDE integrado	No	No	Sí	Sí	Sí
Edición de código avanzada	No / mín.	No / mín.	Sí	Sí	Sí
Compilación cruzada C	Sí	Sí	No	Sí*	Sí*
Soporte múltiples plataformas	No (GB)	Parcial (Z80)	No (ZX Spectrum)	Sí*	Sí*
Depuración integrada	No	No	Sí	Sí*	Sí*
Integración con emuladores	No	No	Sí	Sí*	Sí*
Arquitectura de plugins/extensible	No	No	No	Sí*	Sí*
Curva de aprendizaje	Alta	Alta	Baja-media	Media-alta	Alta
Orientado específicamente a 8 bits	Sí	Sí	Sí	Sí*	Sí*
Enfoque formativo	Medio	Medio	Medio	Variable	Variable
Nota: El asterisco (*) indica que la característica depende de configuración manual.					

Tabla 1.1: Comparativa de herramientas y entornos para desarrollo en plataformas de 8 bits

1.3.1. GBDK-2020

Como se indica en la documentación oficial, GBDK es un kit de desarrollo cruzado para consolas basadas en SM83, Z80 y 6502. Incluye bibliotecas, utilidades de toolchain y el conjunto de compiladores C SDCC [**?**].

En la actualidad, el proyecto se escindió en una rama diferente llamada GBDK-2020 y el proyecto original, aunque aún es posible encontrarlo en internet, no está mantenido por sus autores Pascal Felber, Lars Malmborg, Michael Hope, David Galloway (djmips), John Fuge y colaboradores.

Incluye tanto las librerías necesarias como el compilador para usarlas además de documentación completa.

Un ejemplo de software desarrollado usando este kit es el proyecto Orange FM, cartucho moderno diseño para la familia de consolas portátiles Game Boy que funciona como un sistema de radio estéreo [**?**]. En la Figura 1.1 se muestra la interfaz de este proyecto ejecutándose en la consola Game Boy.

Características de GBDK-2020:

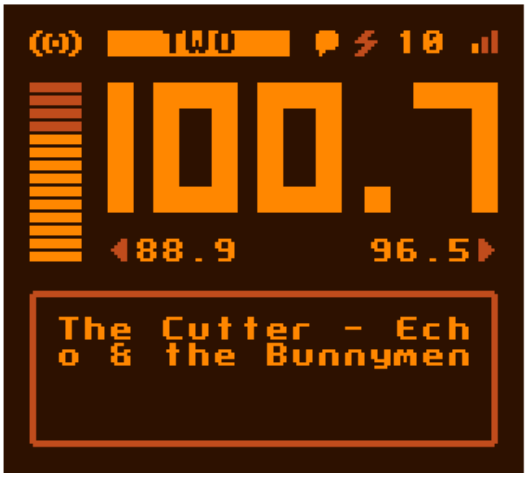

Figura 1.1: Captura de pantalla del proyecto Orange FM.

- Conjunto de herramientas para desarrollo en C y ASM basada en SDCC.
- Múltiples bibliotecas y su código fuente.
- Programas de ejemplo realizados en ASM y en C.
- Soporte para múltiples bancos de memoria ROM.
- La mayor parte del código de las herramientas está bajo la licencia GNU GPL.
- GBDK-2020 permite el desarrollo para las siguientes plataformas
  - Nintendo Game Boy / Game Boy Color.
  - Analogue Pocket.
  - Sega Master System y Game Gear.
  - Mega Duck / Cougar Boy.
  - NES.

Además incluye diversas utilidades para facilitar los desarrollos:

- GBCompress Comprime (y descomprime) datos de archivos binarios con el algoritmo gbcompress.
- png2asset Herramienta para convertir archivos PNG a MetaSprites y mapas de mosaicos en formato GBDK.
- Romusage Utilidad para estimar el uso de ROM de Game Boy y SMS/GG a partir de archivos .noi y .map, ROM binarias y más.

Todo lo anteriormente comentado convierte GBDK-2020 en entorno de desarrollo multiplataforma para sistemas retro de 8 bits, pero sigue dependiendo de un IDE externo o la combinación de editor + terminal para el desarrollo de software.

1.3.2. Z88DK

Z88DK es un kit de desarrollo cruzado para sistemas basados en Z80. Incluye bibliotecas, utilidades de toolchain y el conjunto de compiladores C SDCC.

Fue creado originalmente por Dominic Morris a finales de la década de 1990 con el objetivo de facilitar el desarrollo de software en lenguaje C para el ordenador Cambridge Z88. Con el tiempo, el proyecto ha evolucionado hasta convertirse en un entorno de desarrollo multiplataforma mantenido por una comunidad de desarrolladores que amplían continuamente el soporte para diferentes sistemas basados en procesadores Z80 [**?**].

Como se indica en la documentación oficial: z88dk es el único kit de desarrollo de ensamblador y C que viene listo para usar para crear programas para más de 100 máquinas de la familia z80 (8080, 8085, gbz80, z80, z180, ez80_z80, KC160, Rabbit 2000, 3000, 4000, 5000) [**?**].

El compilador z88dk soporta un gran número de sistemas basados en procesadores Z80 y arquitecturas relacionadas. La tabla 1.2 muestra una selección representativa de algunas de las plataformas más conocidas soportadas por este entorno de desarrollo.

Algunas de las herramientas incluidas en Z88DK:

- zcces la interfaz de la cadena de herramientas. zcc puede generar un binario de salida a partir de cualquier conjunto de archivos fuente de entrada.
- z88dk-scc80 es el compilador C nativo de z88dk.
- z88dk-zsdcc es la personalización de z88dk del compilador C optimizado de sdcc.
- z88dk-z80asm es un ensamblador/vinculador/bibliotecario con todas las funciones que implementa secciones.
- z88dk-zobjcopy permite manipular archivos de objetos y bibliotecas creados por Z80ASM.
- z88dk-appmake procesa los binarios generados por el kit de herramientas en un formato adecuado para equipos de destino específicos. Por ejemplo, puede generar archivos hexadecimales de Intel, cintas, ROM, imágenes de disco, etc.
- z88dk-lib es un instalador de bibliotecas de terceros. Gestiona la instalación, la eliminación y el listado de bibliotecas disponibles.
- z88dk-zx0 y z88dk-zx7 son herramientas de compresión de datos.

ZNAKE [**?**] es un ejemplo de software desarrollado para ZX Spectrum programado en C y compilado con z88dk y compatible con versiones recientes de este compilador, fué publicado en itch.io por marco's retrobits. La Figura 1.2 ilustra una captura de pantalla del juego en ejecución en un emulador de ZX Spectrum.

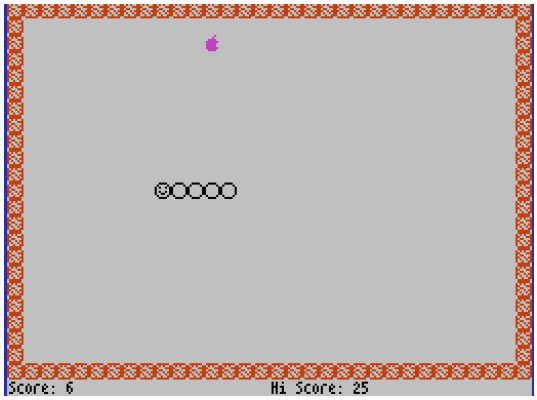

Figura 1.2: Captura de pantalla del proyecto ZNAKE.

Machine	Console	Graphics	File I/O	Sound
Altair 8800	Yes	No	No	No
Amstrad CPC	Yes	640x200	Yes	PSG
Apple II	40x24	No	CP/M	No
Cambridge Z88	Yes	256x64	Yes	1 bit
Colecovision	32x24	256x192	No	PSG
Commodore 128 (Z80)	40x25	640x200	CP/M	SID + PSG
Enterprise 64/128	40x25	672x243	No	Yes
Jupiter Ace	32x24	64x48	No	1 bit
MSX	40x24	256x192	No	PSG
NEC PC-8801	40x25	600x200	No	PSG
Nintendo Game Boy	20x18	160x144	No	Yes
Sega Master System	32x24	256x192	No	PSG
Sharp X1	40x25	640x200	No	PSG
Sinclair ZX80	32x24	64x48	No	1 bit
Sinclair ZX81	32x24	64x48	No	1 bit
ZX Spectrum	32x24	256x192	Yes	1 bit + PSG
ZX Spectrum Next	32x24	256x192	Yes	1 bit + PSG
SAM Coupé	32x24	512x192	No	PSG
TRS-80	64x16	128x48	No	1 bit
TI-83+	16x8	128x64	No	1 bit

Tabla 1.2: Selección representativa de plataformas soportadas por el compilador z88dk. Adaptado de la documentación oficial [**?**].

1.3.3. ZX Basic Studio

ZX Basic Studio [**?**] es un proyecto en desarrollo, iniciado por un desarrollador que utiliza el seudónimo "Dr. Gusman". Se trata de un IDE multiplataforma orientado al compilador Boriel Basic [**?**], un compilador BASIC de código abierto dirigido a usuarios de ZX Spectrum. Boriel Basic permite escribir programas en BASIC que se compilan a código máquina eficiente, ofreciendo facilidad de uso y rendimiento.

El IDE ZX Basic Studio está disponible para Windows, Linux y Mac. Integra un emulador y un depurador, así como un editor de gráficos, basado en una versión mejorada de ZX Graphics desarrollada por Juan Segura Duran ("Duefectu") [**?**]. Además, incluye un editor de ficheros .tap que permite gestionar no solo archivos individuales, sino proyectos completos. Su sistema de plugins amplía las funcionalidades del IDE, incorporando editores de gráficos, coloreadores de sintaxis, tooltips de ayuda y otras herramientas de soporte.

Actualmente, ZX Basic Studio permite compilar y ejecutar programas para ZX Spectrum Next, aunque por el momento la depuración directa en hardware real no está soportada. Dentro del ecosistema ZX Spectrum, ZX Basic Studio se posiciona como una herramienta moderna que combina edición de código, depuración y soporte gráfico, ofreciendo una alternativa más integrada frente a otros IDE tradicionales o emuladores con funcionalidades limitadas. En la Figura 1.3 se puede apreciar la interfaz del IDE con sus diferentes paneles y herramientas integradas.

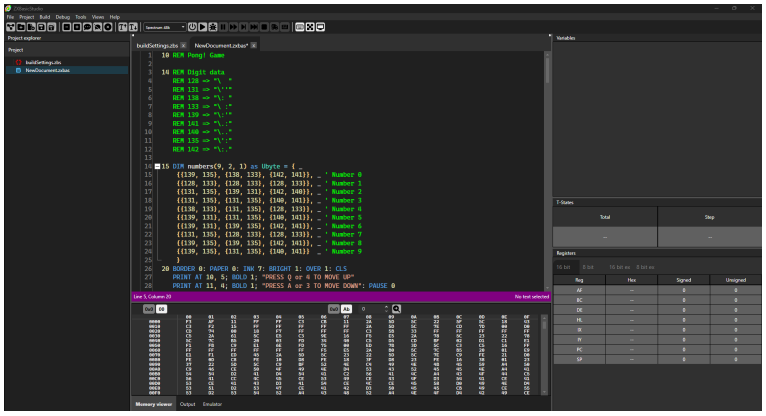

Figura 1.3: Captura de pantalla del proyecto ZX Basic Studio.

1.3.4. VS Code + plugins

Visual Studio Code [**?**] combina la simplicidad de un editor de código fuente con herramientas avanzadas para desarrolladores, como autocompletado de código y depuración mediante IntelliSense. Su diseño minimalista permite que el flujo de edición, compilación y depuración sea ágil, facilitando la concentración en el desarrollo de software.

VS Code es un editor rápido, adecuado para un uso intensivo. Ofrece compatibilidad con la mayoría de los lenguajes de programación, incluyendo resaltado de sintaxis, coincidencia de corchetes, sangría automática, selección de columnas y fragmentos de código reutilizables.

Incluye un depurador interactivo que permite recorrer el código fuente paso a paso, inspeccionar variables, consultar la pila de llamadas y ejecutar comandos en la consola. Además, se integra con múltiples herramientas de compilación y scripting para automatizar tareas rutinarias.

VS Code es compatible con sistemas de control de versiones como Git, proporcionando un visor de diferencias, historial de cambios y gestión de cambios pendientes. La Figura 1.4 muestra la interfaz principal del editor con un proyecto C abierto.

Para el desarrollo específico en plataformas de 8 bits, VS Code cuenta con una amplia gama de extensiones y plugins que facilitan el trabajo con estas arquitecturas. La Tabla 1.3 detalla algunas de las extensiones más relevantes disponibles para este tipo de desarrollo.

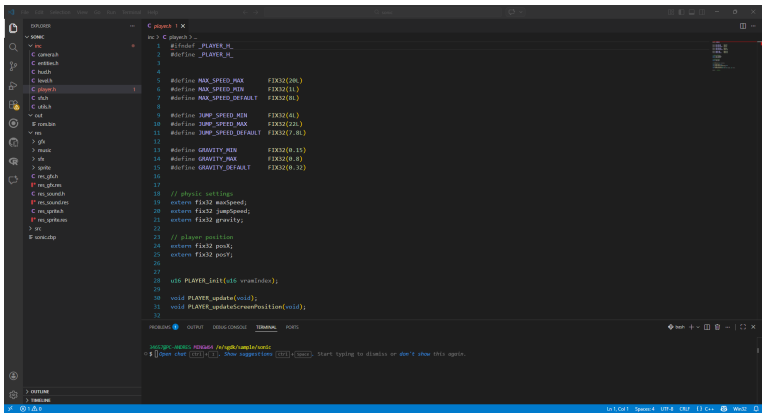

Figura 1.4: Captura de pantalla de Visual Studio Code.

Tabla 1.3: Plugins de VS Code para desarrollo en plataformas de 8 bits

Plugin / Extensión	**Lenguaje / Sistema**	**Función principal**	**Notas**
Retro Assembler	6502, Z80, 6809	Assembler + resaltado de sintaxis	Compatible con NES, C64, ZX Spectrum
vscode-asm6502	6502	Resaltado de sintaxis y snippets	Ideal para NES y Commodore 64
z80-code	Z80	Resaltado de sintaxis y snippets	Compatible con ZX Spectrum, Amstrad CPC
GBDK Support	C (GBDK-2020)	Compilación y snippets para Game Boy	Incluye build tasks y ejemplos
ZX Basic Studio Integration	Boriel Basic	Soporte de sintaxis y proyectos ZX Basic	Requiere configuración manual de paths
C64 Debugger	C64	Depurador integrado para VICE	Funciona con emulador C64
NES Debugger	6502 / NES	Depuración de ROMs	Integración con FCEUX
Arduino / PlatformIO	C / C++	Compilación y depuración hardware	Útil para MCU de 8 bits (AVR, 8051)
GBDK Snippets	C (Game Boy)	Snippets para librerías GBDK	Acelera desarrollo de gráficos y sonido
VIC-20 / C64 Emulation	6502	Integración con emuladores	Ejecuta código en entornos simulados
Retro Debug	Multi Z80 / 6502	Depuración general retro	Soporta snapshots y breakpoints
ZX Spectrum Next Tasks	Z80 / BASIC	Build y ejecución de programas	Compatible con emuladores y hardware real

1.3.5. Eclipse CDT + toolchains

Eclipse es un IDE de código abierto y multiplataforma [?], originalmente diseñado para desarrollo en Java, pero actualmente usado para una amplia variedad de lenguajes y sistemas. Apareció en 2001 como proyecto de IBM y posteriormente pasó a ser administrado por la Eclipse Foundation.

La Figura 1.5 muestra la interfaz de Eclipse CDT con un proyecto C configurado, donde se pueden apreciar los diferentes paneles y herramientas disponibles para el desarrollo.

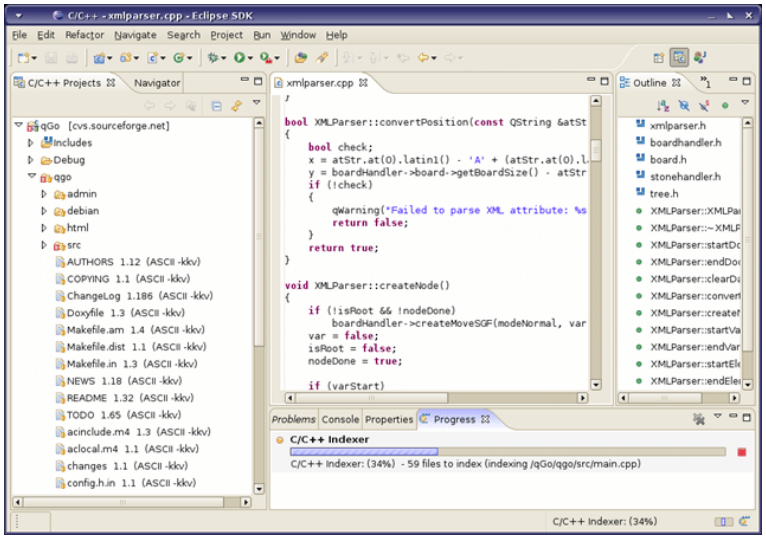

Figura 1.5: Captura de pantalla de Eclipse CDT.

CDT (C/C++ Development Tooling) [?] es un conjunto de extensiones que habilita desarrollo en C y C++, incluyendo compilación, depuración, navegación de código, autocompletado y análisis estático. Eclipse CDT es, por tanto, una versión de Eclipse adaptada para proyectos en C. Por sí solo no incluye toolchains, por lo que debe integrarse con compiladores externos como GCC, SDCC, z88dk o GBDK. Esta modularidad permite su uso en plataformas de 8 bits, aunque requiere configuración manual.

La integración de toolchains externas en Eclipse CDT se realiza mediante la configuración de compiladores y enlazadores dentro del sistema de construcción (Build System). El entorno permite definir rutas a ejecutables, parámetros de compilación, scripts personalizados y perfiles de construcción diferenciados (Debug/Release). En el caso de toolchains como SDCC o z88dk, es necesario definir manualmente los comandos de compilación y enlazado, así como las opciones específicas del microprocesador objetivo.

Entre sus principales ventajas se encuentran la robustez, estabilidad y amplia compatibilidad con toolchains para 8 bits. Sus principales limitaciones son el alto consumo de recursos y la complejidad de configuración inicial.

Eclipse CDT surgió en 2005 como plugin del IDE matriz, permitiendo su aplicación en desarrollo de software embebido y retro gracias a la integración con librerías y herramientas de terceros.

En el contexto específico de desarrollo para sistemas de 8 bits, Eclipse CDT no proporciona asistentes específicos para plataformas retro, por lo que la configuración de entornos como

Game Boy, ZX Spectrum o MSX debe realizarse manualmente. Asimismo, la integración con emuladores requiere scripts externos o configuraciones personalizadas.

La Tabla 1.4 resume las características clave de este entorno de desarrollo para proyectos de 8 bits.

Característica	**Descripción**
IDE completo	Editor, gestión de proyectos y depuración integrada.
Integración toolchains	Compatible con GCC, SDCC, z88dk, GBDK. Múltiples plataformas retro.
Depuración	Soporte GDB y depuradores específicos de MCU/emuladores.
Multiplataforma	Windows, Linux y macOS.
Plugins/Extensible	Soporte para nuevos MCU, lenguajes y emuladores.

Tabla 1.4: Características clave de Eclipse CDT para desarrollo en 8 bits

1.4. Estructura de la memoria

La memoria del proyecto se organiza en varios capítulos que describen de forma progresiva el contexto, desarrollo y resultados del trabajo realizado:

1. *Introducción*: En este apartado se presenta el contexto general del proyecto y la motivación que ha llevado a su desarrollo. Asimismo, se incluye un breve análisis del estado del arte con el fin de situar el trabajo dentro de las soluciones y herramientas existentes.

2. *Análisis*: Se detallan los objetivos del proyecto y las herramientas consideradas para su desarrollo. También se definen los requisitos del sistema y los principales casos de uso que guiarán la implementación.

3. *Diseño*: Se describe el diseño general del sistema, incluyendo la estructura de las pantallas de la aplicación y el flujo de navegación entre ellas.

4. *Implementación*: En este capítulo se explican las funcionalidades desarrolladas y se justifican las principales decisiones técnicas adoptadas durante la implementación.

5. *Pruebas*: Se presenta un conjunto de pruebas destinadas a verificar el correcto funcionamiento del prototipo y a detectar posibles errores en sus funcionalidades principales.

6. *Planificación y presupuesto*: Se incluye la planificación temporal del proyecto mediante un diagrama de Gantt, así como una estimación de los recursos y costes asociados a su desarrollo.

7. *Conclusiones y trabajos futuros*: Finalmente, se recogen las conclusiones derivadas del desarrollo del proyecto, junto con una reflexión sobre los conocimientos adquiridos y posibles líneas de mejora o ampliación para trabajos futuros.